1. **Introduction to Programming**

Definition:

Programming is the process of writing instructions (code) for computers to perform specific tasks. These instructions are written in a programming language, such as Python, which the computer can understand and execute.

Key Concepts:

- **Algorithm**:A step-by-step procedure to solve a problem.
- **Syntax**:Rules that define the structure of the code.
- **Logic**:The sequence of steps to achieve the desired output.

2. **Introduction to Python**

Definition:

Python is a high-level, interpreted, and general-purpose programming language. It is known for its simplicity, readability, and versatility.

Installation and Running:

Jupyter Notebook

- **Definition**:An interactive web-based environment for writing and running Python code.
- **Installation:**

```bash
pip install notebook
```

- **Running**:

```bash
jupyter notebook
```

- **Usage**: Create a new notebook and write Python code in cells.

.py File from Terminal

- **Definition**:A script file containing Python code that can be executed directly.
- **Steps**:
  1. Create a file with a .py extension, e.g., script.py.
  2. Write Python code in the file.
  3. Run the script:

```bash
python script.py
```

Google Colab

- **Definition**:A cloud-based platform that allows you to write and execute Python code in a Jupyter-like environment.
- **Access**:Visit Google Colab.
- **Usage**: Create a new notebook and write Python code in cells.

3. Data Types and Type Conversion

Definition:

- **Data Types**: Categories of data that determine the operations that can be performed on them.
- **Type Conversion**:The process of converting one data type to another.

Python Data Types:

1. **Integer (int)**: Whole numbers, e.g., 10, -5.
2. **Float (float)**: Decimal numbers, e.g., 3.14, -0.5.
3. **String (str)**: Sequence of characters, e.g., "Hello", 'Python'.
4. **Boolean (bool)**: Represents True or False.
5. **List (list)**: Ordered, mutable collection of items, e.g., [1, 2, 3].
6. **Tuple (tuple)**: Ordered, immutable collection of items, e.g., (1, 2, 3).
7. **Set (set)**: Unordered collection of unique items, e.g., {1, 2, 3}.
8. **Dictionary (dict)**: Collection of key-value pairs, e.g., {'name': 'Alice', 'age': 25}.

Type Conversion:

- **Implicit**: Automatic conversion by Python.

```python
a = 10      # int
b = 3.14    # float
c = a + b   # c is float (13.14)
```

- **Explicit**: Manual conversion using functions like int(), float(), str(), etc.

```python
x = "10"
y = int(x)  # Convert string to int
```

4.Variables

Definition:

A variable is a named location in memory used to store data. It acts as a container for values that can be used and manipulated in a program.

Rules for Naming Variables:

1.Must start with a letter or underscore (_).

2.Can only contain alphanumeric characters and underscores.

3.Case-sensitive (myVar and myvar are different).

4.Cannot use Python keywords (e.g., if, else, for).

Example:

```python
name = "Alice"  # String variable
age = 25        # Integer variable
height = 5.6    # Float variable
```

Interview Questions & Answers

Basic Level Questions

Q1. What is programming, and why is it important?

Ans: Programming is the process of writing instructions that a computer can understand to perform specific tasks. It is important because it enables automation, problem-solving, and the development of software applications.

Q2. What is the difference between a compiled and an interpreted language?

Ans:

- **Compiled language:** Translates the entire source code into machine code before execution (e.g., C, C++).
- **Interpreted language:** Executes the code line by line without pre-compilation (e.g., Python, JavaScript).

Q3. What is an algorithm?

Ans: An algorithm is a step-by-step procedure or set of rules designed to solve a problem efficiently.

Q4. What are the fundamental data types in Python?

Ans:

- int (Integer)
- float (Floating-point number)
- str (String)
- bool (Boolean)
- list, tuple, set, dict (Collection types)

Q5. How do you convert one data type into another in Python?

Ans: By using type conversion functions:

- int("10") → Converts string to integer.
- float(5) → Converts integer to float.
- str(25) → Converts integer to string.

Intermediate level Questions

Q6. What is the difference between procedural, object-oriented, and functional programming?

Ans:

- **Procedural Programming:** Based on procedures or routines (e.g., C, Pascal).
- **Object-Oriented Programming (OOP):** Based on objects and classes (e.g., Python, Java).
- **Functional Programming:** Uses pure functions and avoids state changes (e.g., Haskell, Lisp, Python supports functional style).

Q7. What is the difference between running Python interactively and executing a script?

Ans:

- **Interactive mode:** Runs commands line by line in the Python shell (python or python3).
- **Script execution:** Runs an entire .py file using python script.py in the terminal.

Q8. How do you set up a virtual environment in Python? Why is it useful?

Ans:
Use the following commands:

```
sh

python -m venv myenv

source myenv/bin/activate  # On Mac/Linux

myenv\Scripts\activate     # On Windows
```

It helps isolate dependencies between different Python projects.

Q9. What is the difference between shallow copy and deep copy in Python?

Ans:

- **Shallow Copy:** Creates a new object but references nested objects.
- **Deep Copy:** Creates a completely independent copy, including nested objects.

Example:

```
import copy

a = [[1, 2], [3, 4]]

b = copy.copy(a)  # Shallow copy

c = copy.deepcopy(a)  # Deep copy

a[0][0] = 99

print(b)  # [[99, 2], [3, 4]]  (modified)

print(c)  # [[1, 2], [3, 4]] (unchanged)
```

Q10. What is the difference between global, nonlocal, and local variables?

Ans:

- **Local:** Defined inside a function, accessible only within that function.
- **Global:** Declared outside functions and accessible everywhere.
- **Nonlocal:** Used inside nested functions to modify variables from the enclosing function.

Example:

```
def outer():
    x = "outer"
    def inner():
        nonlocal x
        x = "modified"
    inner()
    print(x)  # Output: "modified"
```

Advanced Level Questions

Q11. What is metaprogramming in Python?

Ans: Metaprogramming refers to writing code that manipulates other code (e.g., modifying classes, functions, or code generation at runtime). Python achieves this using **decorators, metaclasses, and reflection** (getattr, setattr, dir).

Example:

```
class Meta(type):
    def __new__(cls, name, bases, dct):
        print(f"Creating class {name}")
        return super().__new__(cls, name, bases, dct)

class MyClass(metaclass=Meta):
    pass

# Output: Creating class MyClass
```

Q12. How does Python handle package dependency conflicts?

Ans: Using **virtual environments** and package managers like pipenv, venv, and conda. Example:

```
sh
pip install virtualenv
virtualenv myenv
source myenv/bin/activate  # On Linux/macOS
myenv\Scripts\activate  # On Windows
```

Q13. What is __pycache__, and why does Python create it?

Ans:

- Python automatically compiles scripts into **bytecode (.pyc files)** stored in the __pycache__ directory.
- This speeds up program execution as Python does not have to recompile .py files every time.

Q14. How does Python handle memory allocation for immutable vs mutable objects?

Ans:

- **Immutable objects** (e.g., int, float, tuple, str) are stored in **constant pools** for efficiency.
- **Mutable objects** (e.g., list, dict, set) are stored on the heap and modified in-place.

Example:

```
a = "hello"
b = "hello"
print(a is b)  # True (same memory reference due to interning)

x = [1, 2, 3]
y = [1, 2, 3]
print(x is y)  # False (separate memory allocation)
```

Q15. What is the difference between del and garbage collection?

Ans:

- del removes a reference to an object but doesn't immediately free memory.
- The **garbage collector (gc)** automatically reclaims memory for unreferenced objects.

Example:

```
import gc

class A:
    def __del__(self):
        print("Object deleted")

obj = A()
del obj  # Calls __del__, but memory may not be freed immediately

gc.collect()  # Forces garbage collection
```